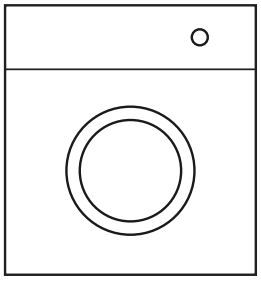
Follow a temperature reading

A pump reports a temperature of 86 °C. One program saves the reading. Another decides whether to record an alert.

From pump to alert



A laundry-room example



Dryer

Someone reads a dryer's temperature and writes it on a slip. The front desk gives one copy to the person keeping the log and another to maintenance.

Maintenance files a report if the temperature is above the limit.

From a sensor to a message

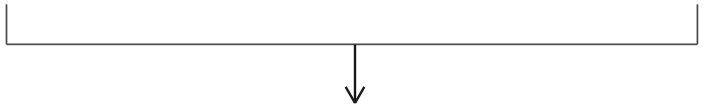
IoT means Internet of Things. A device joins this setup when it sends its measurements over a network. The stream of readings is called telemetry.

Physical sensor

A Linux device reads a connected temperature sensor.

Simulator

A program supplies example temperatures so you can try it without hardware.



Publisher

Either source uses the same message format.

Machine	Time	Temperature	Event ID
pump-01	**12:00 UTC**	**86 °C**	**…af24**

One sample reading, with the event ID shortened.

MQTT and EMQX

MQTT is a set of rules for sending and receiving messages. EMQX is the broker, the program that passes messages to the services that asked for them.

Topic	`machines/pump-01/telemetry`
Payload	Machine, time, temperature and event ID

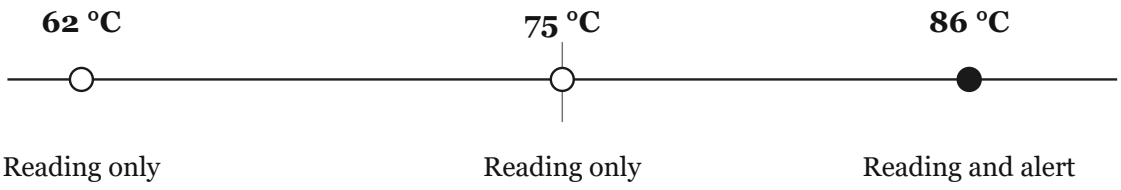
A topic is the label on the slip. A service subscribes to get messages with that label. If a message arrives twice, its event ID lets each service store it once.

What the two services do

A microservice is a program with a specific job that can run separately. Telemetry and Alerts each get the same reading from EMQX.

Telemetry	Alerts
Stores every valid temperature reading in its own database.	Uses its own database to record alerts for temperatures above the limit.

The default limit is 75 °C



Alerts can use a different limit while Telemetry keeps its reading history.

C#, .NET 10 and ASP.NET Core

C#	The language used to write both services. Say 'C sharp'.
.NET 10	The platform that runs the applications.
ASP.NET Core	The part that receives and answers web requests.

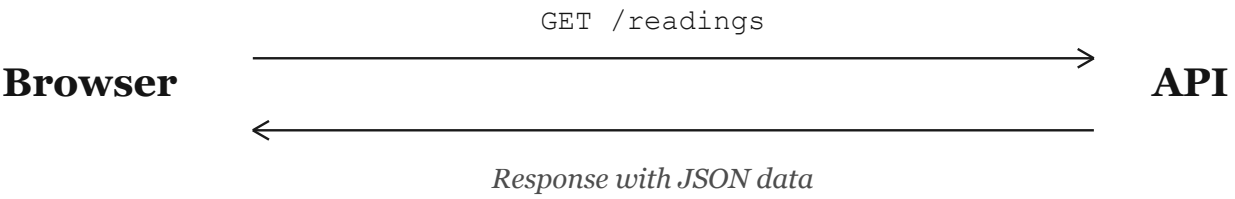
The route in the Telemetry service

```
app.MapGet("/readings", (ReadingStore store) =>
    store.Latest().Select(entry => entry.Reading));
```

MapGet tells ASP.NET Core which C# code to run when the browser asks for /readings. That code fetches the latest measurements.

Read the REST APIs

An API gives another program a way to ask for data. These REST APIs use HTTP, the rules a browser uses to load a page. GET is a request to read something.



Find the same event in both responses

Open the two addresses on the computer running the demo. These examples show selected fields. The event ID is shortened.

GET /readings

`http://localhost:5181/readings`

```
[
  {
    "id": "…af24",
    "temperatureC": 86
  }
]
```

GET /alerts

`http://localhost:5182/alerts`

```
[
  {
    "reading": { "id": "…af24" },
    "thresholdC": 75
  }
]
```

Try it

If the reading were 75 °C, where would it appear? In /readings. The Alerts service records temperatures above the limit.